

# **“VotingHub – Online Voting System with Aadhaar-Style OTP Verification”**

A Project Report Submitted in the partial fulfillment of the requirements for the award of the  
degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

|                        |                   |
|------------------------|-------------------|
| <b>KEERTHANA G</b>     | <b>2520090132</b> |
| <b>THOTA JHANA SRI</b> | <b>2520090121</b> |
| <b>SUTTE ALEKHYA</b>   | <b>2520090126</b> |

Under the Esteemed Guidance of

**Dr. Sk. Kaja Shareef**

**Head of the Department,CS&IT**

**Department of Computer Science and Information Technology**

**DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT**

**(25CS1302E) | Section 12 | Team 9**

**Academic Year 2026–2027 | Trimester 4**



**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University**  
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



**Declaration**

The Project Report entitled **“VotingHub – Online Voting System with Aadhaar-Style OTP Verification”** is a record of bonafide work of **KEERTHANA G – 2520090132, THOTA JHANA SRI – 2520090121, SUTTE ALEKHYA – 2520090126** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 9, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

**KEERTHANA G – 2520090132**  
**THOTA JHANA SRI – 2520090121**  
**SUTTE ALEKHYA – 2520090126**

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



## **CERTIFICATE**

This is to certify that the project report entitled **“VotingHub – Online Voting System with Aadhaar-Style OTP Verification”** is a bonafide work done and submitted by **KEERTHANA G – 2520090132, THOTA JHANA SRI – 2520090121, SUTTE ALEKHYA – 2520090126** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 9, at KLH University during 2026–2027, Trimester 4.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## **ACKNOWLEDGEMENT**

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us in one way or another in completing this project.

Our greatest appreciation goes to our Course Coordinator, Ramatenki Sateesh Kumar, and to our guide, Dr. Sk. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for his tremendous support, encouragement and guidance throughout this project.

We express our gratitude to Dr. Sk. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

**KEERTHANA G – 2520090132**  
**THOTA JHANA SRI – 2520090121**  
**SUTTE ALEKHYA – 2520090126**

## ABSTRACT

VotingHub is a full-stack, web-based online voting system designed to provide a secure, controlled and privacy-aware digital voting platform for institutional and organisational elections. Traditional and manual voting depends on physical presence, paper ballots and manual counting, which makes the process slow, error-prone and difficult to audit. At the same time, a naive online voting application can allow impersonation, repeated voting and tampering with stored votes. VotingHub addresses these concerns through a database-driven design in which voter identity, election eligibility, voting authorization and ballot storage are modelled as separate, constrained relations in a MySQL database.

The system allows voters to register, verify their identity through a one-time password (OTP), log in, view elections, check their eligibility, vote for a candidate and review their voting-related information. Administrators can create and manage elections, candidates, voters, election status, results and audit information. The OTP mechanism follows an Aadhaar-style flow purely as an academic demonstration. The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.

The application is built with a React and Vite frontend, a Java 21 and Spring Boot backend, Spring Data JPA for persistence, and MySQL 8.0 as the relational datastore. Components communicate through REST APIs. Passwords are protected with BCrypt hashing, access is controlled through authentication and authorization checks, and duplicate voting is prevented by several cooperating layers: authentication, election-status validation, eligibility validation, a voting-authorization record with a used flag, authoritative backend validation, a unique election-user constraint and transaction-based processing. The design separates voter authorization data from ballot data for privacy awareness; it does not claim to be a cryptographically certified secret-ballot system.

To demonstrate distributed backend development, the system is also decomposed into three Spring Boot microservices – an Auth Service (port 8081), an Election Service (port 8082) and a Voting Service (port 8083) – each owning its own logical database (votehub\_auth, votehub\_election and votehub\_voting). The Voting Service obtains election information from the Election Service through REST calls and never reads another service's database directly. The project illustrates relational schema design, ER modelling, keys and constraints, REST API design, authentication, OTP verification, authorization, duplicate-vote prevention, microservices and full-stack development, and results in a self-hostable system that runs locally on a standard development machine.

# INDEX

| S.No. | Chapters                 | Topics                                                                                                                                                                   | Page No. |
|-------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|       | Acknowledgement          |                                                                                                                                                                          | 4        |
|       | Abstract                 |                                                                                                                                                                          | 5        |
| 1     | Introduction             | 1.1 Background of the Project<br>1.2 Problem Statement<br>1.3 Scope of the Project<br>1.4 Objectives<br>1.5 Importance of the Application<br>1.6 Target Users / Audience | 7        |
| 2     | System Requirements      | 2.1 Hardware Requirements<br>2.2 Software Requirements<br>2.3 Development Tools and Frameworks                                                                           | 9        |
| 3     | Technology Stack         | 3.1 Frontend<br>3.2 Backend<br>3.3 Database<br>3.4 Version Control<br>3.5 Authentication and Security                                                                    | 10       |
| 4     | System Architecture      | 4.1 High-Level Architecture Diagram<br>4.2 Description of Each Layer<br>4.3 Deployment Architecture                                                                      | 12       |
| 5     | Design                   | 5.1 Database Design Principles<br>5.2 ER Diagram / Database Schema<br>5.3 Design Decisions and Assumptions<br>5.4 Privacy-Aware Voting Design                            | 14       |
| 6     | Implementation           | 6.1 Module-wise Implementation<br>6.2 Frontend Logic<br>6.3 API Integration<br>6.4 Authentication and Authorization                                                      | 18       |
| 7     | Features                 | 7.1 List of Main Features<br>7.2 How Each Feature Works                                                                                                                  | 22       |
| 8     | Testing                  | 8.1 Unit / Functional Testing<br>8.2 Integration Testing<br>8.3 Tools Used for Testing<br>8.4 Test Cases and Results                                                     | 25       |
| 9     | Deployment               | 9.1 Steps to Deploy<br>9.2 Environment Configuration<br>9.3 Hosting / Local Deployment                                                                                   | 27       |
| 10    | Challenges & Limitations | 10.1 Issues Faced During Development<br>10.2 Solutions Applied<br>10.3 Current Limitations / Known Issues                                                                | 29       |
| 11    | Future Enhancements      | 11.1 Planned Features<br>11.2 Possible Integrations or Optimizations                                                                                                     | 31       |
| 12    | Conclusion               | 12.1 Summary of the Project<br>12.2 What Was Achieved<br>12.3 Skills Learned                                                                                             | 32       |
| 13    | References               | Documentation, tools, and resources used                                                                                                                                 | 33       |
| 14    | Appendices               | 14.1 Important Code Snippets<br>14.2 Installation / Setup Instructions<br>14.3 API Reference Summary<br>14.4 Database SQL Scripts<br>14.5 Screenshots                    | 34       |

# CHAPTER 1: INTRODUCTION

## 1.1 Background of the Project

Elections in colleges, student councils, clubs, cooperative societies and small organisations are still frequently conducted through paper ballots or ad-hoc spreadsheets. Manual voting requires voters to be physically present at a fixed time and place, depends on staff to verify identities and count ballots by hand, and leaves very little durable evidence that can be inspected after the event. Counting errors, misplaced ballots, queues and disputes over who actually voted are common practical problems. Digital platforms promise convenience, but a poorly designed online system introduces new risks such as impersonation, repeated voting and silent modification of stored results.

VotingHub was developed as a database-centred answer to these problems. It is a full-stack web application in which voter accounts, elections, candidates, voting authorizations, ballots and audit records are stored in a relational MySQL database with explicit keys and constraints, and in which every sensitive decision – whether a person may vote, whether an election is open, whether a vote has already been cast – is taken by the backend rather than by the user interface. Voters verify their identity with a one-time password before they can log in, and administrators manage the complete election lifecycle from a dedicated dashboard.

The project is also the practical component of the course Database Systems Engineering and Distributed Backend Development. For this reason the system is deliberately structured to exercise both halves of the course: relational schema design, SQL, transactions and integrity constraints on the database side, and REST-based service decomposition into Auth, Election and Voting microservices on the distributed backend side.

*The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.*

## 1.2 Problem Statement

Given a population of registered voters and a set of elections, each with its own candidates and its own opening and closing time, the system must answer four questions reliably: how can a voter's identity be verified before access is granted; how can the system guarantee that each eligible voter can vote at most once in each election even under repeated or concurrent requests; how can ballots be stored so that the record of who was authorized to vote is kept separate from the record of what was voted; and how can administrators manage elections and inspect results and system activity without being able to alter votes silently. Manual voting trades away speed, accessibility and auditability, while careless online voting trades away integrity and privacy. VotingHub is designed to keep all of these properties within the limits of an academic project.

## 1.3 Scope of the Project

The system covers voter registration and admin registration; OTP-based identity verification; secure login with BCrypt-hashed passwords; a voter dashboard and an admin dashboard; creation and management of elections and candidates; election status handling; eligibility

verification and voting authorization; vote casting with duplicate-vote prevention; voting history; notifications; aggregate result retrieval; and audit logs. The backend is available both as a main Spring Boot application and as three cooperating microservices (Auth, Election and Voting) that illustrate distributed backend design with database-per-service ownership.

The following were treated as out of scope for this submission: real Aadhaar or UIDAI authentication, biometric verification, a cryptographically certified secret-ballot protocol (for example blind signatures or homomorphic tallying), mobile applications, large-scale load handling, and use in legally binding public elections. The system is an academic demonstration and should not be used for statutory elections.

## **1.4 Objectives**

- Provide a web application through which voters can register, verify their identity using an OTP, log in, view elections, check eligibility and cast a vote for a candidate.
- Provide administrators with the ability to create and manage elections, candidates, voters, election status, results and audit information.
- Design a normalised relational schema in MySQL 8.0 with primary keys, foreign keys, unique constraints and indexes that enforce data integrity inside the database itself.
- Prevent duplicate voting through layered controls so that the guarantee does not depend on the user interface alone.
- Separate voter authorization information from ballot information as a privacy-aware design choice.
- Secure accounts with BCrypt password hashing, OTP verification with expiry and attempt limits, and role-based authorization.
- Demonstrate distributed backend development by decomposing the backend into Auth, Election and Voting microservices that own separate logical databases and communicate through REST APIs.
- Maintain audit logs and election events so that important actions can be reviewed afterwards.

## **1.5 Importance of the Application**

Voting is an activity in which trust matters more than convenience. An online system is useful only if participants can believe that only eligible persons vote, that each person votes once, and that the counted result reflects the votes cast. By enforcing these rules in the backend and in database constraints, VotingHub removes the dependence on manual supervision and produces an inspectable record of activity. Administrators gain a single place to run elections, and voters gain a simple interface that tells them clearly which elections are open to them. For the authors, the project is a practical study of relational design, transactional processing, authentication, REST API design and microservice decomposition.

## **1.6 Target Users / Audience**

The system is intended for institutions and small organisations such as colleges, student councils, clubs, associations and cooperative societies that need a simple, controlled platform for internal elections. Two roles are supported. A Voter can register, verify an OTP, log in, view elections, check eligibility, vote and view personal voting history and notifications. An Administrator can create and manage elections and candidates, change election status, manage voters, view aggregate results and review audit logs.

## CHAPTER 2: SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

VotingHub is light enough to run its complete stack – the React frontend, the Spring Boot backend services and the MySQL server – on a single development machine. Running several Spring Boot services together alongside a browser, an IDE and MySQL is the most demanding configuration, which is why 8 GB of RAM is recommended. Table 2.1 summarises the hardware requirements.

*Table 2.1: Hardware Requirements*

| Component    | Minimum                                                                                  | Recommended                                                   |
|--------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Processor    | Dual-core processor                                                                      | Quad-core or better                                           |
| Memory (RAM) | 4 GB                                                                                     | 8 GB (when all services, MySQL, IDE and browser run together) |
| Storage      | Sufficient free space for JDK, Node modules, Maven repository, MySQL data (about 3–5 GB) | SSD with 10 GB free                                           |
| Network      | Internet/network connection for dependency download and browser access                   | Stable broadband connection                                   |

### 2.2 Software Requirements

The software required to build, run and test the system is listed in Table 2.2. All of it is freely available for academic use.

*Table 2.2: Software Requirements*

| Software                         | Purpose                                                            |
|----------------------------------|--------------------------------------------------------------------|
| Windows / Linux operating system | Development and execution environment                              |
| Java 21 (JDK)                    | Runtime and compiler for the Spring Boot backend and microservices |
| Apache Maven                     | Build and dependency management for the Java projects              |
| Node.js and npm                  | Runtime and package manager for the React + Vite frontend          |
| MySQL 8.0                        | Relational database server                                         |
| Modern web browser               | Accessing the voter and admin interfaces                           |
| Git and GitHub                   | Version control and collaboration                                  |
| VS Code / IntelliJ IDEA          | Code editors / IDEs                                                |
| MySQL Workbench                  | Database design, SQL execution and data inspection                 |
| Postman                          | Manual testing of REST APIs of all services                        |

### 2.3 Development Tools and Frameworks

The web client was built with React and Vite using JavaScript, with Axios for HTTP calls and React Router for navigation. The backend was developed in Java 21 with Spring Boot and Spring Data JPA, built with Maven. MySQL Workbench was used to design tables, run SQL scripts and inspect stored data, and Postman was used to exercise the REST APIs of the Auth,

Election and Voting services independently of the frontend. Git and GitHub were used for source control and for sharing work among the three team members, with VS Code and IntelliJ IDEA as the primary editors.

## CHAPTER 3: TECHNOLOGY STACK

Table 3.1 gives an overview of the technologies used in VotingHub; the sections that follow explain the role of each in the project.

*Table 3.1: Technology Stack*

| Layer           | Technology                                     | Role in the project                                        |
|-----------------|------------------------------------------------|------------------------------------------------------------|
| Frontend        | React, Vite, JavaScript, Axios, React Router   | User interface, routing and REST API consumption           |
| Backend         | Java 21, Spring Boot, Spring Data JPA, Maven   | Business logic, validation, REST APIs and persistence      |
| Database        | MySQL 8.0                                      | Relational storage with keys, constraints and indexes      |
| Communication   | REST APIs (JSON over HTTP)                     | Frontend–service and service–service communication         |
| Security        | BCrypt, OTP verification, authorization checks | Password protection, identity verification, access control |
| Version control | Git, GitHub                                    | Source management and collaboration                        |
| Testing tools   | Postman, MySQL Workbench, browser tools        | API testing and data verification                          |

### 3.1 Frontend

The frontend is a single-page application built with **React**, a component-based JavaScript library in which the interface is composed from reusable components such as login forms, election cards, candidate lists and dashboard panels. **Vite** is used as the build tool and development server; it provides fast start-up and hot module replacement during development and serves the application on port 5174. The code is written in **JavaScript**. **React Router** maps URLs to pages (home, login, registration, OTP verification, voter dashboard, admin dashboard and so on) and allows protected routes that redirect unauthenticated users. **Axios** is the HTTP client used to call the REST APIs of the backend services, and it centralises request configuration and error handling.

### 3.2 Backend

The backend is written in **Java 21** using **Spring Boot**, which provides embedded server support, dependency injection and auto-configuration. Each functional area is organised in the usual layered style: controllers expose REST endpoints, services hold business rules, and repositories handle persistence. **Spring Data JPA** maps Java entities to MySQL tables and generates repository implementations, so that common queries and transactional operations can be written without hand-coded JDBC. **Maven** manages dependencies and builds. The backend exists in two forms: a main application that runs on port 8080 and a set of three microservices – Auth Service on 8081, Election Service on 8082 and Voting Service on 8083.

### 3.3 Database

**MySQL 8.0** is the datastore. It is a relational database management system with ACID transactions, foreign keys, unique constraints and indexes, all of which VotingHub relies on. The schema contains twelve tables: `users`, `otp_verifications`, `elections`, `candidates`, `voting_authorizations`, `votes`, `voting_history`, `sessions`, `notifications`, `election_events`, `system_settings` and `audit_logs`. In the microservices architecture the data is divided into three logical databases – `votehub_auth`, `votehub_election` and `votehub_voting` – each owned by one service. MySQL Workbench was used for modelling, script execution and inspection.

### 3.4 Version Control

The project source is managed with **Git** and hosted on **GitHub**. Version control allowed the three team members to work on the frontend, the main backend and the individual microservices in parallel, to track the history of changes, and to recover earlier versions when an experiment failed. Sensitive configuration such as database credentials is supplied through environment variables and is not stored in the repository.

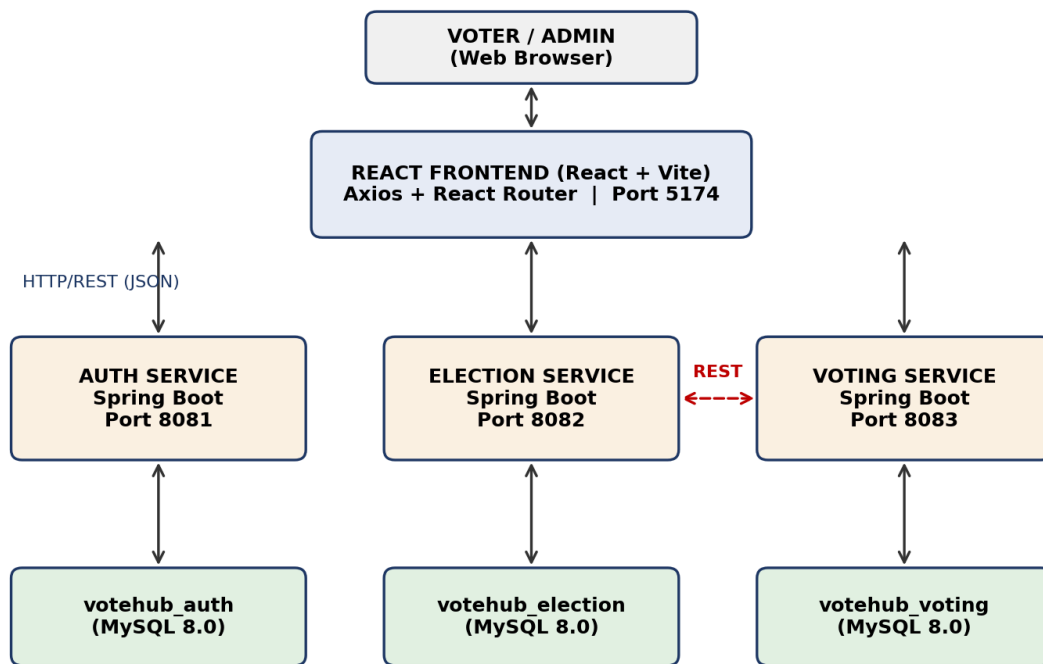
### 3.5 Authentication and Security

Passwords are never stored in plain text; they are hashed with **BCrypt**, an adaptive hashing algorithm that incorporates a salt and a configurable work factor. Identity verification uses an **OTP** (one-time password) with an expiry time and a limited number of attempts, recorded in the `otp_verifications` table. The OTP flow imitates the shape of an Aadhaar-style verification for demonstration purposes only. The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure. Authenticated sessions are recorded in the `sessions` table, and every protected operation is authorised on the server according to the user's role and the state of the election. Chapter 6 describes these mechanisms in more detail.

## CHAPTER 4: SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture Diagram

VotingHub follows a layered client–server architecture in which a React frontend communicates over REST with backend services, and each backend service stores its data in MySQL. For the microservices component of the course (CO6), the backend is decomposed into three Spring Boot services, each with its own logical database. Figure 4.1 shows the voter or administrator using the React frontend on port 5174, the Auth Service (8081), Election Service (8082) and Voting Service (8083), and the databases `votehub_auth`, `votehub_election` and `votehub_voting`.



*Each service owns its logical database. A service never reads another service's database; cross-service data is obtained only through REST APIs (e.g. Voting Service -> Election Service).*

Figure 4.1: High-Level System Architecture

### 4.2 Description of Each Layer

**Presentation layer.** The React frontend is the only component that users interact with. It renders the voter and admin interfaces, collects input, and calls the backend services through Axios. It contains no authoritative security logic: it may hide or disable controls for convenience, but every rule is enforced again by the backend.

**Auth Service (port 8081).** The Auth Service owns user identity. It handles voter and admin registration, OTP generation and verification, login and session handling, and stores its data in the `votehub_auth` database (users, otp\_verifications and sessions). Passwords are hashed with BCrypt before storage.

**Election Service (port 8082).** The Election Service owns election data. It provides the creation, update and retrieval of elections and candidates and the management of election status and election events. Its data resides in the `votehub_election` database.

**Voting Service (port 8083).** The Voting Service owns the voting process: eligibility checks, voting authorizations, vote casting, duplicate-vote prevention, voting history and aggregate results. Its data resides in the `votehub_voting` database. To confirm that an election exists, is active and that a candidate belongs to it, the Voting Service calls the Election Service through its REST API. It does not connect to the `votehub_election` database.

**Main backend (port 8080).** The project also contains the original, consolidated Spring Boot backend that implements the application over the complete twelve-table schema. It forms the functional baseline from which the three microservices were derived and runs on port 8080. The microservices are described in this report as the distributed-backend (CO6) design; the report does not claim any capability for either form beyond what the project implements.

**Data layer.** MySQL 8.0 stores all persistent data. Spring Data JPA provides the mapping between Java entities and tables, and the integrity rules – primary keys, foreign keys, unique constraints and indexes – are defined in the database schema so that they hold regardless of which application component writes the data.

#### ***Database ownership and service communication***

Each microservice is the sole owner of its logical database. No service executes SQL against another service's database; instead, data that belongs to another service is requested through that service's REST API. This rule keeps services loosely coupled, lets each one change its internal schema independently, and makes the boundaries of responsibility explicit. For example, when a vote is submitted the Voting Service sends a REST request to the Election Service to check the election's status and the candidate, and then writes only to `votehub_voting`.

### **4.3 Deployment Architecture**

The system is deployed locally as separate processes: the React development server on port 5174, the main backend on port 8080, the Auth Service on port 8081, the Election Service on port 8082, the Voting Service on port 8083, and a MySQL 8.0 server on its default port. Each Spring Boot application is started independently with Maven, and database credentials are supplied through environment variables. Chapter 9 describes the steps in detail. The report does not describe a cloud or container deployment because none is part of the current implementation; these are listed as future enhancements in Chapter 11.

## CHAPTER 5: DESIGN

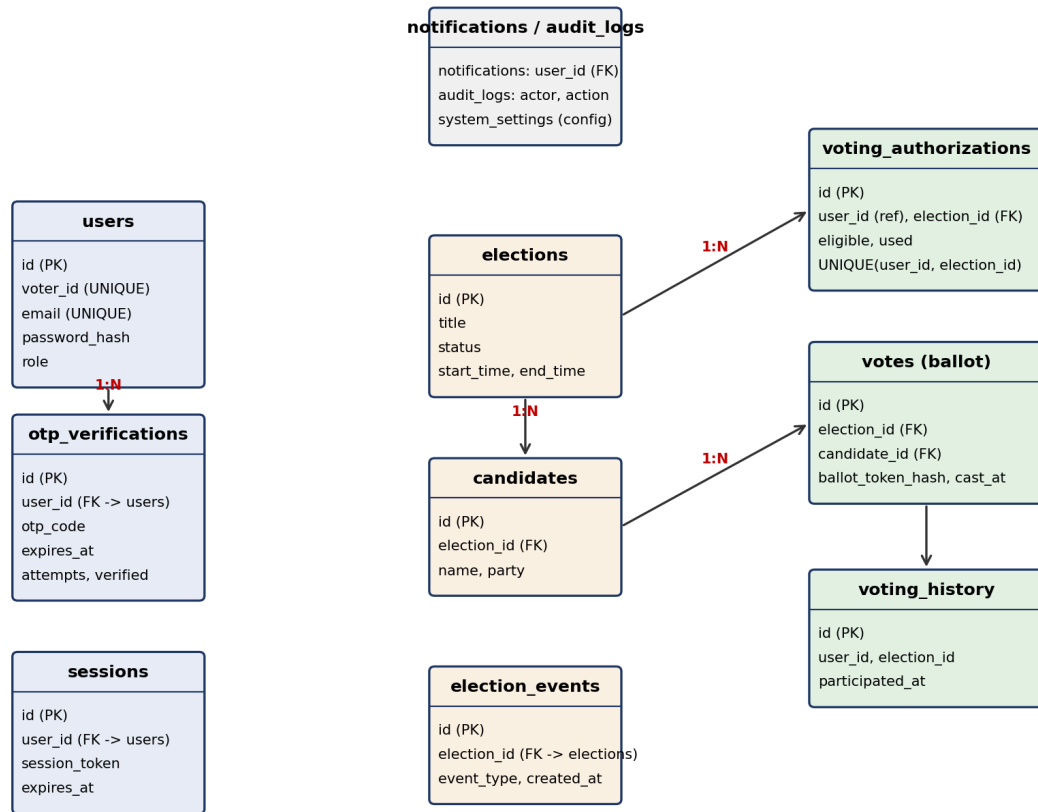
### 5.1 Database Design Principles

The schema keeps each fact in one place and expresses relationships through identifier references. Every table has a surrogate primary key; relationships between tables are declared with foreign keys; and business rules that must never be violated – one account per voter ID, one account per e-mail address, one voting authorization per user per election – are declared as unique constraints so that the database itself rejects violations, even if an application bug or a repeated request attempts to create them. Compound or single-column indexes are placed on the columns that are actually used for lookups, such as the election and user identifiers on authorization and history rows and the election identifier on candidates and ballots.

Design follows the usual normalisation guidelines: user details are stored once in users and referenced elsewhere by id; candidate details are stored once in candidates and referenced by ballots; and election details are stored once in elections. Time-dependent facts (OTP expiry, election start and end time, ballot timestamp) are stored as timestamps so that validity can be checked in the backend. A second guiding rule is that authorization and ballot information are kept in different relations, as explained in Section 5.4.

### 5.2 ER Diagram / Database Schema

The core data model is summarised in Figure 5.1. A user (voter or admin) may have many OTP verification records and many sessions. An election has many candidates and many election events. A voting authorization links a user to an election and records eligibility and whether it has been used. A ballot (row of votes) links an election to the chosen candidate together with a ballot token hash and a timestamp. A voting-history row records that a user participated in an election. Notifications and audit logs relate to users and system activity, and system\_settings holds configuration values.



Entity-relationship overview (core entities). Votes hold no user\_id: authorization and ballot are separated.

Figure 5.1: Entity Relationship Diagram

Table 5.1 describes the purpose of each table. The column lists in Figure 5.1 and in Appendix 14.4 show the principal attributes of the design.

Table 5.1: Database Tables

| Table                 | Purpose                                                        | Key and relationship notes                                    |
|-----------------------|----------------------------------------------------------------|---------------------------------------------------------------|
| users                 | Voter and admin account information, hashed password, role     | PK id; unique voter ID and e-mail                             |
| otp_verifications     | OTP code, expiry, attempt count and verified status            | PK id; FK user_id → users                                     |
| elections             | Election title, description, status, start and end time        | PK id                                                         |
| candidates            | Candidates standing in an election                             | PK id; FK election_id → elections                             |
| voting_authorizations | Voter eligibility for an election and whether it has been used | PK id; unique (user_id, election_id)                          |
| votes                 | Ballot: election, candidate, ballot token hash, timestamp      | PK id; FK election_id, candidate_id; no voter identity column |
| voting_history        | Record that a user participated in an election                 | PK id; refers to user and election                            |
| sessions              | Authentication/session information                             | PK id; FK user_id → users                                     |
| notifications         | Messages shown to users                                        | PK id; FK user_id → users                                     |
| election_events       | Election lifecycle events (created, opened, closed)            | PK id; FK election_id → elections                             |

| Table           | Purpose                                          | Key and relationship notes   |
|-----------------|--------------------------------------------------|------------------------------|
| system_settings | System configuration key–value pairs             | PK id / unique setting key   |
| audit_logs      | Records of system activity and important actions | PK id; refers to acting user |

### *Purpose of each table*

- **users** stores voter and admin information such as voter ID, name, e-mail, role and the BCrypt password hash.
- **otp\_verifications** supports OTP verification: it records the generated code, its expiry time, the number of failed attempts and whether verification succeeded.
- **elections** stores election information, including its status (for example upcoming, active or closed) and its schedule.
- **candidates** stores the candidates that belong to a particular election.
- **voting\_authorizations** stores voter eligibility for an election and a used status that is set once the voter has voted.
- **votes** stores ballot information – the election, the chosen candidate, a ballot token hash and a timestamp.
- **voting\_history** stores participation history, so that a voter can see which elections they have taken part in without revealing their choice.
- **sessions** stores authentication and session information for logged-in users.
- **notifications** stores notifications for users.
- **election\_events** stores election lifecycle events.
- **system\_settings** stores system configuration.
- **audit\_logs** stores records of system activity.

### *Logical databases in the microservices architecture*

In the microservices design the data is divided into three logical databases, as shown in Table 5.2. Each database belongs to exactly one service. Where one service needs information belonging to another (for instance the status of an election), it asks the owning service through a REST API instead of reading the other database. Cross-service references such as a user id stored in the voting database are therefore logical references by identifier, not database-level foreign keys across databases.

**Table 5.2: Logical Databases and Ownership**

| Logical database | Owner service           | Principal data                                         |
|------------------|-------------------------|--------------------------------------------------------|
| votehub_auth     | Auth Service (8081)     | users, OTP verification records, sessions              |
| votehub_election | Election Service (8082) | elections, candidates, election events                 |
| votehub_voting   | Voting Service (8083)   | voting authorizations, ballots (votes), voting history |

The cross-cutting tables notifications, system\_settings and audit\_logs form part of the complete twelve-table schema used by the main backend (port 8080).

### 5.3 Design Decisions and Assumptions

- **Backend validation is authoritative.** Whether a user may vote is decided by the backend using the database state. Disabling a button in the React interface is a convenience for the user and is not treated as a security mechanism.
- **Constraints in the database.** Uniqueness of voter ID, e-mail and (user, election) authorization is declared in the schema so that integrity does not depend only on application code.
- **One authorization per voter per election.** A voting authorization carries a used flag; once a vote is recorded the flag is set and further attempts are rejected.
- **Separation of authorization and ballot.** The ballot table does not store the voter's identity (Section 5.4).
- **Database per service.** Each microservice owns its logical database, and cross-service data is obtained through REST APIs.
- **Simulated OTP.** The OTP flow is an academic simulation of an Aadhaar-style check. It is not connected to UIDAI or any real Aadhaar authentication infrastructure.
- **Credentials outside the code.** Database usernames and passwords are provided through environment variables and are not committed to the repository.
- **Assumption on users.** It is assumed that voters are registered members of the organisation holding the election and that an administrator assigns or confirms eligibility.

### 5.4 Privacy-Aware Voting Design

A central design decision is that voter authorization information and ballot information are separated. Conceptually, the voting authorization answers the question “is this person permitted to vote in this election, and have they already done so?”, while the ballot answers the question “which candidate was chosen in this election?”. The two relations are deliberately not joined by a voter identity column, as summarised below.

| VOTING AUTHORIZATION | BALLOT            |
|----------------------|-------------------|
| User ID              | Election ID       |
| Election ID          | Candidate ID      |
| Eligibility          | Ballot Token Hash |
| Used status          | Timestamp         |

Because the ballot row carries a ballot token hash rather than a user id, a casual inspection of the votes table does not directly show who voted for whom, and aggregate results can be computed from ballots alone. The authorization table, in contrast, shows who was allowed to vote and whether they used that permission, which is what the duplicate-vote check needs.

*The design separates voter authorization from ballot information for privacy awareness. This project does not claim to provide a cryptographically certified secret-ballot system.*

The limits of this approach should be stated plainly. Anyone with full administrative control of the database and application server, and with access to timing information, may be able to

correlate records. Strong ballot secrecy and end-to-end verifiability require cryptographic protocols that are beyond the scope of this project and are listed under future enhancements.

## CHAPTER 6: IMPLEMENTATION

### 6.1 Module-wise Implementation

The system is organised into functional modules. Each module consists of React pages and components, REST endpoints in a Spring Boot controller, business logic in a service class, and JPA entities and repositories that persist data in MySQL.

#### ***Voter Registration***

A voter submits personal details including a voter ID and e-mail address. The Auth Service validates the input, rejects a duplicate voter ID or e-mail (checked in code and enforced again by unique constraints), hashes the password with BCrypt, and stores the new user. Registration then triggers the OTP step.

#### ***Admin Registration***

Administrator accounts are created through a separate registration path that assigns the admin role. Role information stored in the users table is later used for authorization, so that admin-only operations such as election creation can be refused for ordinary voters.

#### ***OTP Verification***

The backend generates a one-time password for the user and records it in otp\_verifications with an expiry time and an attempt counter. When the user submits the code, the backend checks that it matches, has not expired and that the maximum number of attempts has not been exceeded; on success the account is marked verified. An invalid, expired or over-attempted OTP is rejected. The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.

#### ***Login***

Login accepts credentials, compares the supplied password with the stored BCrypt hash, confirms that the account is verified, and creates a session record. An invalid login returns an error without revealing which part of the credentials was wrong.

#### ***Voter Dashboard***

After login the voter dashboard lists the elections visible to the voter and links to voting history and notifications. Data is retrieved with Axios from the relevant services.

#### ***Election Management***

Administrators create, edit and delete elections, supplying title, description and schedule. The Election Service stores these in the elections table and records lifecycle events in election\_events.

#### ***Candidate Management***

Administrators add, edit and remove candidates for an election. Each candidate row carries the identifier of the election to which it belongs, and candidates are retrieved per election.

### ***Election Status***

An election moves through lifecycle states such as upcoming, active and closed. Voting is accepted only while an election is active; the status is checked by the backend at the time a vote is submitted.

### ***Eligibility Verification***

Before a voter may proceed, the backend checks whether the voter is eligible for the selected election. The result is shown to the voter, but the check is repeated on the server when a vote is actually submitted.

### ***Voting Authorization***

A voting authorization record ties a user to an election and carries an eligibility flag and a used flag. A voter can vote only if an unused authorization exists. The unique constraint on (user, election) guarantees that there is at most one such record.

### ***Vote Casting***

The voter selects a candidate and submits the vote. The Voting Service validates the request, confirms with the Election Service that the election is active and the candidate belongs to it, and then stores the ballot and updates the authorization within one transaction.

### ***Duplicate Vote Prevention***

Duplicate voting is prevented by several layers working together, described in Section 7.2 (Feature 13), so that no single layer is solely responsible.

### ***Voting History***

After voting, a row is written to `voting_history`. The voter can view the elections in which they participated; the history shows participation, not the candidate chosen.

### ***Notifications***

Notifications inform users of events such as successful verification, election openings or a recorded vote. They are stored in the notifications table and fetched for display on the dashboard.

### ***Results***

Results are computed as aggregate counts of ballots per candidate for an election. Because the ballot table does not hold voter identity, aggregation does not expose individual choices.

### ***Audit Logs***

Important actions – registration, verification, login, election changes and vote recording – create audit log entries that administrators can review.

## **6.2 Frontend Logic**

The frontend is a React application built with Vite. Pages are implemented as functional components with state managed by React hooks. React Router defines routes for the public pages (home, login, registration, OTP verification), the voter pages (dashboard, election list, voting page, history, notifications) and the admin pages (dashboard, election management,

candidate management, results, audit logs). Protected routes check whether the user is logged in and has the required role before rendering a page. Forms validate input on the client for a better user experience, but the backend repeats every validation. After a vote is submitted the interface shows a confirmation or the error returned by the server. Buttons such as “Vote” may be disabled after a successful vote to avoid accidental double clicks, but this is only a usability aid: the decision to accept or reject a vote always rests with the backend.

### 6.3 API Integration

The frontend communicates with the backend services using Axios over HTTP with JSON request and response bodies. Requests for registration, OTP verification and login are directed to the Auth Service (port 8081); requests for elections and candidates to the Election Service (port 8082); and requests for eligibility, voting, history and results to the Voting Service (port 8083). Cross-origin requests from the Vite development server (port 5174) are allowed by CORS configuration on the services. A summary of the endpoints is given in Appendix 14.3.

Service-to-service communication also uses REST. When the Voting Service processes a vote, it calls the Election Service to confirm the election and candidate before accepting the ballot. Figure 6.1 shows the complete voting workflow.

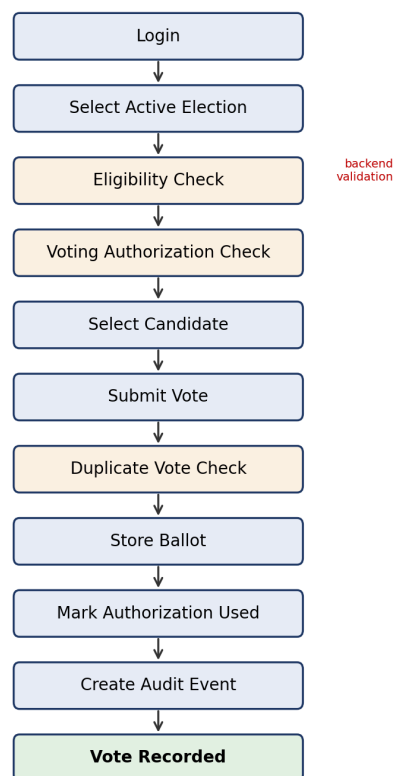


Figure 6.1: Voting Workflow

The steps are as follows. After login, the voter selects an active election. The system performs an eligibility check and a voting-authorization check. The voter selects a candidate and submits the vote. The backend then performs the duplicate-vote check, stores the ballot, marks the

authorization as used and creates an audit event, after which the vote is recorded. The backend validation at each of these stages is authoritative; the frontend only presents the result.

Database integration uses Spring Data JPA. Entities annotated for the tables are accessed through repository interfaces; operations that must succeed or fail together, such as storing a ballot and marking an authorization used, are placed in transactional service methods so that a failure leaves no partial state.

## **6.4 Authentication and Authorization**

Registration creates the account, hashes the password with BCrypt and starts OTP verification. Login verifies the password hash and the verified status and then creates a session. Authorization is role- and state-based: only administrators may create or modify elections and candidates and view audit logs; only authenticated, verified voters may vote; voting is allowed only while an election is active, only if the voter is eligible and holds an unused voting authorization, and at most once per election. Every one of these checks is performed in the backend on each request, independent of what the frontend displays.

The OTP mechanism deserves a specific statement. It reproduces the user-facing pattern of an Aadhaar-style one-time password for demonstration only. The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.

## CHAPTER 7: FEATURES

### 7.1 List of Main Features

1. Voter Registration
2. Admin Registration
3. OTP Verification
4. Secure Login
5. Voter Dashboard
6. Admin Dashboard
7. Election Management
8. Candidate Management
9. Election Status
10. Voter Eligibility
11. Voting Authorization
12. Vote Casting
13. Duplicate Vote Prevention
14. Voting History
15. Notifications
16. Aggregate Results
17. Audit Logs
18. Privacy-Aware Ballot Storage
19. Microservices Communication

### 7.2 How Each Feature Works

#### ***1. Voter Registration***

The voter fills in a registration form. The Auth Service checks that the voter ID and e-mail are not already registered, hashes the password with BCrypt and saves the user with the voter role. Duplicate voter IDs and e-mail addresses are rejected both by validation code and by unique constraints in the database.

#### ***2. Admin Registration***

An administrator account is registered through a separate path that sets the admin role. The role controls access to election, candidate, results and audit functions.

#### ***3. OTP Verification***

After registration the system issues an OTP with an expiry time. The user enters the code; the backend checks it against the stored value, the expiry and the attempt limit, and marks the account verified on success. Too many wrong attempts or an expired code cause rejection and require a new OTP. The Aadhaar-style OTP verification implemented in this project is a

simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.

#### ***4. Secure Login***

Login compares the submitted password with the stored BCrypt hash and checks that the account has been verified. A session record is created and the user is directed to the voter or admin dashboard according to role.

#### ***5. Voter Dashboard***

The voter dashboard presents available elections, election status, eligibility information, voting history and notifications in one place.

#### ***6. Admin Dashboard***

The admin dashboard provides access to election management, candidate management, voter management, results and audit logs.

#### ***7. Election Management***

Administrators create elections with title, description and schedule, edit them and delete them where allowed. The Election Service persists them and records lifecycle events.

#### ***8. Candidate Management***

Candidates are added to a specific election and can be edited or removed by administrators. The candidate list shown to voters is fetched per election.

#### ***9. Election Status***

The status of an election (for example upcoming, active, closed) determines whether voting is possible. A status change is recorded as an election event. A vote submitted to a closed or not-yet-open election is rejected by the backend.

#### ***10. Voter Eligibility***

Eligibility is evaluated for each voter and election. An ineligible voter is informed and any attempt to vote is rejected on the server.

#### ***11. Voting Authorization***

Eligibility is represented by a voting authorization row for the (user, election) pair, with an eligibility flag and a used flag. Voting requires an eligible, unused authorization.

#### ***12. Vote Casting***

The voter chooses a candidate and submits the vote. The following sequence is executed by the backend:

1. Authenticate the request and identify the user.
2. Confirm with the Election Service that the election is active and the candidate belongs to it.
3. Check eligibility and the voting authorization.
4. Check that the authorization has not been used.

5. Store the ballot (election, candidate, ballot token hash, timestamp).
6. Mark the authorization as used and write voting history.
7. Create an audit event.

Steps 5 to 7 are executed within a transaction where applicable so that either all of them take effect or none do.

### ***13. Duplicate Vote Prevention***

Duplicate voting is controlled by multiple layers, summarised in Table 7.1. No single layer is relied on alone.

***Table 7.1: Layers of Duplicate Vote Prevention***

| Layer                             | Role                                                                                         |
|-----------------------------------|----------------------------------------------------------------------------------------------|
| Authentication                    | Only a logged-in, verified user can reach the voting operation                               |
| Election status validation        | Only an active election accepts votes                                                        |
| Eligibility validation            | Only eligible voters may proceed                                                             |
| Voting authorization              | A voter needs an authorization record for the election                                       |
| Used flag                         | The authorization is marked used once the vote is recorded; later attempts fail              |
| Backend validation                | All checks are enforced on the server and are authoritative                                  |
| Database constraints              | Constraints reject invalid or duplicate rows at the data level                               |
| Unique election-user relationship | At most one authorization exists per user per election                                       |
| Transaction-based processing      | Ballot storage and marking the authorization used succeed or fail together, where applicable |

Frontend button disabling is intentionally not part of this list, since a user could bypass the interface and call the API directly; it is only a convenience for honest users.

### ***14. Voting History***

Voting history records participation in elections and is shown to the voter. It records that the voter took part, not which candidate was chosen.

### ***15. Notifications***

Users receive notifications about events that concern them; these are read from the notifications table.

### ***16. Aggregate Results***

Results are retrieved as counts per candidate for an election. Because only aggregate figures are presented, no individual ballot is attributed to a voter in the result view.

### ***17. Audit Logs***

Administrators can review audit logs to see important system activity, such as registrations, logins, election changes and vote recording.

### ***18. Privacy-Aware Ballot Storage***

Ballot rows contain the election, the candidate, a ballot token hash and a timestamp, while authorization rows contain the user and used status. The separation reduces the direct linkage between a person and a ballot. It is a privacy-aware design and not a certified secret-ballot system.

### ***19. Microservices Communication***

The Auth, Election and Voting services run independently on ports 8081, 8082 and 8083. The Voting Service calls the Election Service via REST to validate elections and candidates. No service accesses another service's database.

## CHAPTER 8: TESTING

### 8.1 Unit / Functional Testing

Functional testing verified that each module behaves according to its specification: registration and duplicate detection, OTP verification with invalid, expired and over-attempted codes, login, election and candidate management, eligibility checks, vote casting, duplicate-vote rejection and result retrieval. Test cases were designed around the rules enforced by the backend, since these – rather than the visual behaviour of the interface – protect the integrity of the election.

### 8.2 Integration Testing

Integration testing concentrates on the interaction between the frontend and the services and between the services themselves. The Auth, Election and Voting services were exercised through their REST APIs, and the communication in which the Voting Service obtains election information from the Election Service was tested as a separate case. Database state was inspected in MySQL Workbench to confirm that rows such as ballots, authorizations and history were created or left unchanged as expected.

### 8.3 Tools Used for Testing

- **Postman** – sending requests to the Auth, Election and Voting service APIs and inspecting responses.
- **MySQL Workbench** – running SQL queries to verify stored data and constraints.
- **Web browser and developer tools** – manual end-to-end testing of the React interface and inspection of network calls.
- **Maven** – building and running the Spring Boot applications.

### 8.4 Test Cases and Results

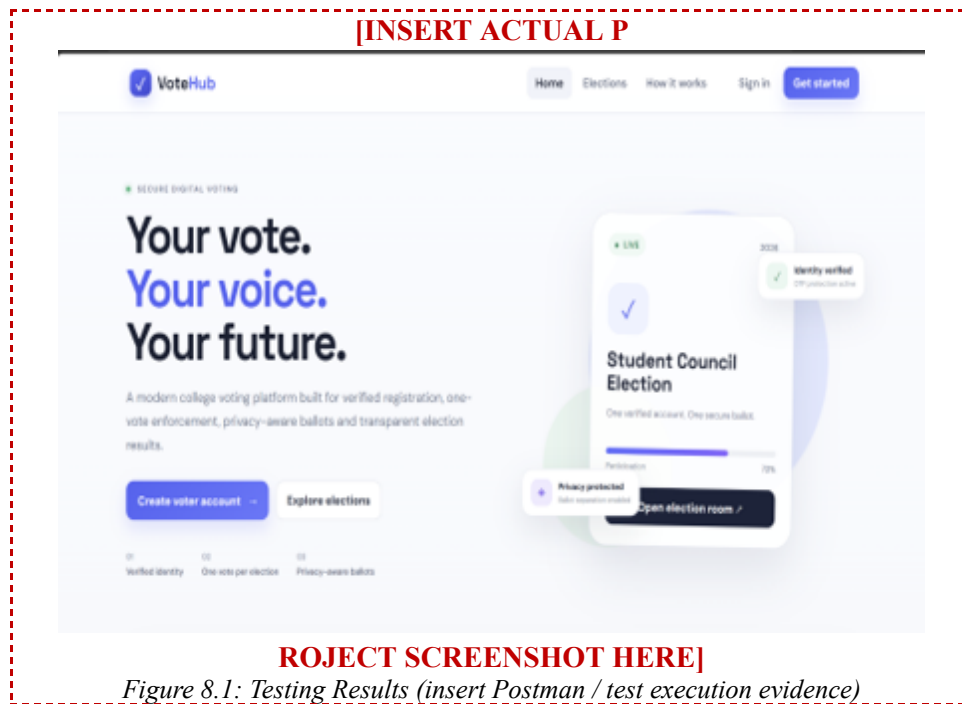
Table 8.1 lists the test cases defined for VotingHub together with the expected result. The actual result of each case is to be filled in from the final test execution; no result is claimed here before it has been recorded.

*Table 8.1: Test Cases*

| # | Test Case            | Expected Result                         | Actual Result   |
|---|----------------------|-----------------------------------------|-----------------|
| 1 | Valid registration   | Account created; OTP step starts        | To be confirmed |
| 2 | Duplicate voter ID   | Registration rejected with error        | To be confirmed |
| 3 | Duplicate e-mail     | Registration rejected with error        | To be confirmed |
| 4 | Invalid OTP          | Verification rejected; attempt counted  | To be confirmed |
| 5 | Expired OTP          | Verification rejected; new OTP required | To be confirmed |
| 6 | Maximum OTP attempts | Further attempts blocked                | To be confirmed |
| 7 | Successful login     | Session created; dashboard shown        | To be confirmed |
| 8 | Invalid login        | Login rejected with error               | To be confirmed |
| 9 | Election retrieval   | Elections returned                      | To be confirmed |

| #  | Test Case                            | Expected Result                                   | Actual Result   |
|----|--------------------------------------|---------------------------------------------------|-----------------|
| 10 | Candidate retrieval                  | Candidates of the election returned               | To be confirmed |
| 11 | Eligibility check                    | Eligible voter proceeds; ineligible voter refused | To be confirmed |
| 12 | Valid vote                           | Ballot stored; authorization marked used          | To be confirmed |
| 13 | Duplicate vote                       | Second vote rejected                              | To be confirmed |
| 14 | Voting in closed election            | Vote rejected                                     | To be confirmed |
| 15 | Unauthorized voting                  | Vote rejected                                     | To be confirmed |
| 16 | Results retrieval                    | Aggregate counts returned                         | To be confirmed |
| 17 | Admin election creation              | Election created; non-admin refused               | To be confirmed |
| 18 | Candidate management                 | Candidates added, edited and removed              | To be confirmed |
| 19 | Auth Service API                     | Endpoints on port 8081 respond correctly          | To be confirmed |
| 20 | Election Service API                 | Endpoints on port 8082 respond correctly          | To be confirmed |
| 21 | Voting Service API                   | Endpoints on port 8083 respond correctly          | To be confirmed |
| 22 | Voting → Election REST communication | Voting Service obtains election data through REST | To be confirmed |

*Test execution result to be updated after final demonstration. The results in Table 8.1 have not been recorded yet and must not be read as passed.*



*Figure 8.1: Testing Results (insert Postman / test execution evidence)*

*Figure 8.1: Testing Results*

## CHAPTER 9: DEPLOYMENT

### 9.1 Steps to Deploy

VotingHub is deployed locally. All components can be run on a single machine. The required software is Java 21, Maven, Node.js, npm and MySQL 8.0. The steps are as follows.

1. Install Java 21, Maven, Node.js with npm, and MySQL 8.0, and verify them from the command line.
2. Start the MySQL server and create the logical databases (votehub\_auth, votehub\_election and votehub\_voting for the microservices, and the main application database for the main backend) using the SQL scripts of Appendix 14.4 or the schema generation configured in the project.
3. Set the database environment variables (Section 9.2) in the shell that will start each backend application.
4. Start the main backend on port 8080, if it is required, with Maven (for example `mvn spring-boot:run` in its project directory).
5. Start the Auth Service (port 8081), the Election Service (port 8082) and the Voting Service (port 8083), each from its own project directory and terminal.
6. In the frontend directory run `npm install` once, then start the Vite development server, which serves the application on port 5174.
7. Open the frontend in a browser, register a voter, verify the OTP, log in and try the voting workflow. Use Postman to test each service API directly.

**Table 9.1: Application Ports**

| Component                  | Port                                     |
|----------------------------|------------------------------------------|
| Frontend (React + Vite)    | 5174                                     |
| Main Backend (Spring Boot) | 8080                                     |
| Auth Service               | 8081                                     |
| Election Service           | 8082                                     |
| Voting Service             | 8083                                     |
| MySQL 8.0                  | Default MySQL port (3306 unless changed) |

### 9.2 Environment Configuration

Database connection details are not hard-coded. Each Spring Boot application reads its database username and password from environment variables, with the connection URL pointing to the logical database owned by that application. Real passwords must never be committed to Git or placed in this report. Placeholders are shown below.

```
DB_USERNAME=root
DB_PASSWORD=<your-password>
```

Each service uses its own database name in its connection URL, for example `votehub_auth` for the Auth Service, `votehub_election` for the Election Service and `votehub_voting` for the Voting Service. The Voting Service additionally needs the base URL of the Election Service (`http://localhost:8082`) so that it can call it over REST. The frontend is configured with the base URLs of the services, and the backend allows the frontend origin (`http://localhost:5174`) through CORS configuration.

### **9.3 Hosting / Local Deployment**

The current system is hosted locally for demonstration. Every service is a separate Java process and the frontend is served by the Vite development server. No cloud hosting, container orchestration or public URL is claimed. For a production-style deployment the frontend would be built into static files and served by a web server or reverse proxy, the services would run as managed processes or containers, and MySQL would run on a secured, backed-up server. These improvements are listed under future enhancements in Chapter 11.

## CHAPTER 10: CHALLENGES & LIMITATIONS

### 10.1 Issues Faced During Development

The development of a multi-component system exposed typical integration problems. The challenges below describe the kinds of difficulty that arise in a project of this architecture and that were addressed during development.

- **Database connectivity.** Configuring the connection from each Spring Boot application to MySQL (correct URL, database name, username and password) required care, particularly with several services and databases.
- **CORS configuration.** Requests from the Vite server on port 5174 to services on other ports were blocked by the browser until the allowed origins were configured.
- **Port conflicts.** Running the frontend, the main backend and three services at once required a unique port for each (5174, 8080, 8081, 8082, 8083).
- **Microservice configuration.** Each service needed its own configuration, its own database and the address of the services it depends on.
- **REST communication between services.** The Voting Service had to call the Election Service and handle the cases in which the election was missing, closed or unreachable.
- **OTP handling.** Expiry, attempt counting and the verified status had to be handled correctly for valid, invalid, expired and over-attempted codes.
- **Duplicate vote protection.** Preventing repeated votes required more than a user-interface check, including the used flag, uniqueness constraints and transactional processing.
- **Database relationships.** Defining keys and relationships, and deciding which relationships become logical references across service databases.
- **Authentication.** Combining BCrypt password checks, OTP verification and session handling into a coherent login flow.
- **Environment configuration.** Keeping credentials out of the source code while allowing every team member to run the project.

### 10.2 Solutions Applied

Database credentials were moved to environment variables and the connection URL of each application was set to its own logical database. CORS was configured on the backend to permit the frontend origin. A fixed port was assigned to each component and recorded in the setup instructions. Each microservice was given its own configuration and communicates with others only through REST APIs, with explicit handling of error responses. The OTP logic stores an expiry and an attempt counter and rejects verification when either condition fails. Duplicate-vote prevention was moved into the backend and the database through the authorization record, the used flag, a unique (user, election) constraint and transactional processing, so that it holds even

if the frontend is bypassed. Postman was used extensively to test each service in isolation before connecting the frontend.

### **10.3 Current Limitations / Known Issues**

- The Aadhaar-style OTP is a simulation and provides no real identity assurance. The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.
- The privacy-aware ballot design separates authorization from ballot, but is not a cryptographically certified secret-ballot system and does not offer end-to-end verifiability.
- The system is intended for academic demonstration and is not suitable for statutory or high-stakes elections.
- Services are deployed locally; there is no API gateway, service discovery, central monitoring or container-based deployment.
- Authentication is session-based; token-based mechanisms such as JWT are not part of the current implementation.
- Results are retrieved by request and are not pushed to clients in real time.
- Performance under heavy concurrent load has not been evaluated.
- The test results in Chapter 8 remain to be confirmed after the final demonstration.

## CHAPTER 11: FUTURE ENHANCEMENTS

### 11.1 Planned Features

- Real-time election updates using WebSocket or Server-Sent Events (SSE), so that dashboards and results update without refreshing.
- JWT-based authentication to provide stateless, token-based access control across the services.
- Stronger identity-provider integration to replace the simulated OTP with a genuine, authorised verification mechanism.
- Stronger cryptographic secret-ballot mechanisms, such as blind signatures or verifiable tallying, to provide ballot secrecy beyond the privacy-aware separation used now.
- Improved audit monitoring, including searchable, tamper-evident audit trails and alerts.
- A mobile application for voters.

### 11.2 Possible Integrations or Optimizations

- Docker-based containerisation of all services and the database for reproducible deployment.
- Cloud deployment of the frontend, services and a managed MySQL instance.
- An API Gateway as a single entry point that handles routing, authentication and rate limiting.
- Service Discovery so that services locate each other dynamically rather than through fixed addresses.
- Enhanced monitoring and logging of service health and performance.
- Redis caching of frequently requested data such as election and candidate lists.
- Automated unit, integration and end-to-end tests integrated into a continuous-integration pipeline.

## CHAPTER 12: CONCLUSION

### 12.1 Summary of the Project

This project set out to build an online voting system in which the important rules of an election are enforced by the backend and the database rather than by the user interface. VotingHub consists of a React and Vite frontend, a Java 21 and Spring Boot backend using Spring Data JPA, and a MySQL 8.0 database, communicating through REST APIs. The backend is also decomposed into Auth, Election and Voting microservices with separate logical databases, demonstrating distributed backend design.

### 12.2 What Was Achieved

The project delivers voter and admin registration, OTP verification, secure login with BCrypt-hashed passwords, voter and admin dashboards, election and candidate management, election status handling, eligibility verification, voting authorization, vote casting with layered duplicate-vote prevention, voting history, notifications, aggregate results and audit logs. The database design separates voter authorization from ballot information for privacy awareness. The microservices architecture shows service ownership of data and REST-based communication, with the Voting Service obtaining election information from the Election Service. The OTP mechanism is a simulated academic demonstration and no real UIDAI or Aadhaar integration is claimed. The final test execution results are to be updated after the demonstration.

### 12.3 Skills Learned

The project demonstrates and reinforced the following skills:

- **Database engineering with MySQL and SQL** – creating schemas, writing queries and inspecting data.
- **Relational database design and ER modelling** – identifying entities, relationships, primary keys and foreign keys.
- **Constraints and integrity** – using unique constraints and indexes to enforce rules in the database.
- **REST APIs and Spring Boot** – designing controllers, services and repositories with Spring Data JPA.
- **React** – building a component-based frontend with routing and API integration.
- **Authentication, OTP verification and authorization** – BCrypt hashing, expiry and attempt limits, and role-based access control.
- **Duplicate vote prevention** – combining layered validation, constraints and transactions.
- **Microservices** – database-per-service ownership and REST communication between services.
- **Git and GitHub** – collaborative version control.

- **Testing and full-stack development** – designing test cases, using Postman and connecting frontend, backend and database.

## CHAPTER 13: REFERENCES

- Oracle — Java Platform, Standard Edition 21 Documentation, <https://docs.oracle.com/en/java/javase/21/>
- Spring Boot — Official Documentation, <https://spring.io/projects/spring-boot>
- Spring Data JPA — Official Documentation, <https://spring.io/projects/spring-data-jpa>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- React — Official Documentation, <https://react.dev>
- React Router — Official Documentation, <https://reactrouter.com>
- Vite — Official Documentation, <https://vite.dev>
- Axios — Official Documentation, <https://axios-http.com/docs/intro>
- Apache Maven — Official Documentation, <https://maven.apache.org/guides/>
- Git — Official Documentation, <https://git-scm.com/doc>
- GitHub Docs, <https://docs.github.com>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- MDN Web Docs — HTTP and REST-related documentation, <https://developer.mozilla.org>
- OWASP Password Storage Cheat Sheet (hashing and BCrypt), [https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- Postman Learning Center, <https://learning.postman.com>

## CHAPTER 14: APPENDICES

### 14.1 Important Code Snippets

*The snippets below are illustrative of the logic described in this report. Replace or reconcile them with the exact source code in the project repository before submission.*

#### ***Snippet A: Vote casting service (illustrative)***

The method runs inside a transaction so that storing the ballot and marking the authorization used succeed or fail together. The checks are performed in the backend and are authoritative.

```
@Transactional
public VoteResponse castVote(Long userId, Long electionId, Long candidateId) {
    // 1. Election must be active (obtained from Election Service via REST)
    ElectionDto election = electionClient.getElection(electionId);
    if (!"ACTIVE".equals(election.getStatus()))
        throw new ElectionClosedException();

    // 2. Authorization must exist, be eligible, and be unused
    VotingAuthorization auth = authorizationRepo
        .findByUserIdAndElectionId(userId, electionId)
        .orElseThrow(UnauthorizedVoterException::new);
    if (!auth.isEligible() || auth.isUsed())
        throw new DuplicateOrIneligibleVoteException();

    // 3. Store ballot (no user id) and mark authorization used
    Vote ballot = new Vote(electionId, candidateId, tokenHash(), Instant.now());
    voteRepo.save(ballot);
    auth.setUsed(true);
    authorizationRepo.save(auth);
    historyRepo.save(new VotingHistory(userId, electionId));
    auditService.record("VOTE_RECORDED", userId, electionId);
    return VoteResponse.recorded();
}
```

#### ***Snippet B: Password hashing with BCrypt (illustrative)***

```
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();
user.setPasswordHash(encoder.encode(request.getPassword()));
// login: encoder.matches(rawPassword, user.getPasswordHash())
```

#### ***Snippet C: Frontend API call with Axios (illustrative)***

```
const res = await axios.post(`${VOTING_URL}/api/voting/vote`, {
    electionId, candidateId,
});
// the server decides whether the vote is accepted
```

### 14.2 Installation / Setup Instructions

1. Install Java 21, Maven, Node.js with npm, MySQL 8.0 and Git.
2. Clone the project repository from GitHub.
3. Start MySQL and create the databases using the scripts in Section 14.4.
4. Set DB\_USERNAME=root and DB\_PASSWORD=<your-password> as environment variables (use your own credentials; never commit them).
5. Start the main backend (port 8080) if required, then the Auth (8081), Election (8082) and Voting (8083) services using Maven.
6. In the frontend directory run npm install and start the Vite server (port 5174).

7. Open the application in a browser and register a voter. The OTP flow is a simulation. The Aadhaar-style OTP verification implemented in this project is a simulated academic mechanism and is NOT connected to UIDAI or any real Aadhaar authentication infrastructure.

### 14.3 API Reference Summary

*Representative summary of the REST endpoints. Confirm exact paths and methods against the controllers and the Postman collection of the project.*

**Table 14.1: API Reference**

| Service         | Endpoint (representative)            | Purpose                             |
|-----------------|--------------------------------------|-------------------------------------|
| Auth (8081)     | POST /api/auth/register              | Register voter or admin             |
| Auth (8081)     | POST /api/auth/otp/verify            | Verify OTP                          |
| Auth (8081)     | POST /api/auth/login                 | Authenticate and create session     |
| Election (8082) | GET /api/elections                   | List elections                      |
| Election (8082) | POST /api/elections                  | Create election (admin)             |
| Election (8082) | PATCH /api/elections/{id}/status     | Change election status (admin)      |
| Election (8082) | GET /api/elections/{id}/candidates   | List candidates of an election      |
| Election (8082) | POST /api/elections/{id}/candidates  | Add candidate (admin)               |
| Voting (8083)   | GET /api/voting/eligibility          | Check eligibility and authorization |
| Voting (8083)   | POST /api/voting/vote                | Cast a vote                         |
| Voting (8083)   | GET /api/voting/history              | Voting history of a user            |
| Voting (8083)   | GET /api/voting/results/{electionId} | Aggregate results                   |

### 14.4 Database SQL Scripts

*Representative DDL showing the key tables, keys and constraints. Reconcile column names and types with the project's actual SQL scripts.*

```
CREATE DATABASE votehub_auth; CREATE DATABASE votehub_election; CREATE DATABASE
votehub_voting;

-- votehub_auth
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  voter_id VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(120) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(20) NOT NULL,
  verified BOOLEAN DEFAULT FALSE
);
CREATE TABLE otp_verifications (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  otp_code VARCHAR(10) NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  attempts INT DEFAULT 0,
  verified BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

-- votehub_election
CREATE TABLE elections (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  status VARCHAR(20) NOT NULL,
```

```

    start_time TIMESTAMP, end_time TIMESTAMP
);
CREATE TABLE candidates (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    election_id BIGINT NOT NULL,
    name VARCHAR(120) NOT NULL,
    FOREIGN KEY (election_id) REFERENCES elections(id)
);

-- votehub_voting
CREATE TABLE voting_authorizations (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    election_id BIGINT NOT NULL,
    eligible BOOLEAN DEFAULT TRUE,
    used BOOLEAN DEFAULT FALSE,
    UNIQUE (user_id, election_id)
);
CREATE TABLE votes (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    election_id BIGINT NOT NULL,
    candidate_id BIGINT NOT NULL,
    ballot_token_hash VARCHAR(128) NOT NULL,
    cast_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

The remaining tables (sessions, notifications, election\_events, system\_settings, audit\_logs, voting\_history) follow the same conventions: a surrogate primary key, foreign keys where the referenced table is in the same database, and indexes on the columns used for lookup.

## 14.5 Screenshots

Figure 14.1 lists the application screenshots to be included. No screenshot has been fabricated; each placeholder must be replaced by an actual capture from the running VotingHub project.

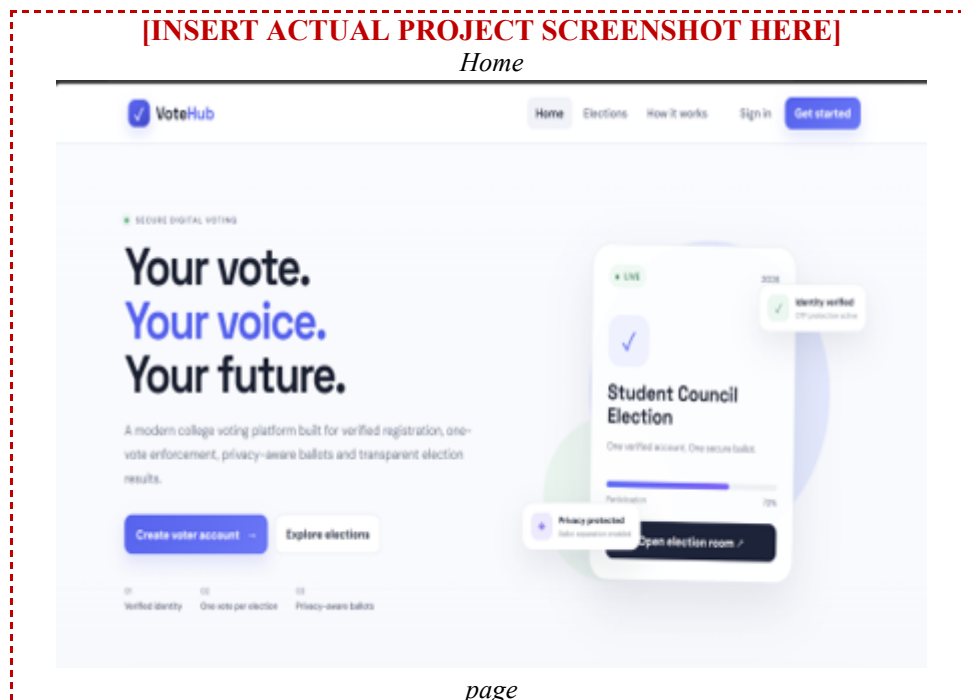


Figure 14.1(a): Home page

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

### Login

**VoteHub** Home Elections How it works Sign in Get started

**Secure access starts here.**

Verified identity, protected sessions and a clear path from registration to ballot.

- ✓ OTP verification
- ✓ Secure password storage
- ✓ One-vote enforcement

**SECURE SIGN IN**

Welcome back

Sign in with the account you created on VoteHub.

Voter ID / Employee ID

jphns

Password

.....

Sign in

New here? [Create voter account](#)

Figure 14.1(b): Login

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

### Voter registrati

**VoteHub** Home Elections How it works Sign in Get started

**Secure access starts here.**

Verified identity, protected sessions and a clear path from registration to ballot.

- ✓ OTP verification
- ✓ Secure password storage
- ✓ One-vote enforcement

**VOTER REGISTRATION**

Create your voter account

Register securely to access active college elections.

Voter ID

e.g. VTR202600I

Full name

Your full name

Date of birth

dd-mm-yyyy

Aadhaar number

12-digit number

Email address

name@example.com

Mobile number

10-digit mobile

Password

Create a strong password

Confirm password

Repeat password

☐ I agree to the VoteHub terms and understand this is a college/varsity voting system.

Create account

Already verified? [Sign in](#)

on

Figure 14.1(c): Voter registration

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

OTP verification

Figure 14.1(d): OTP verification

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

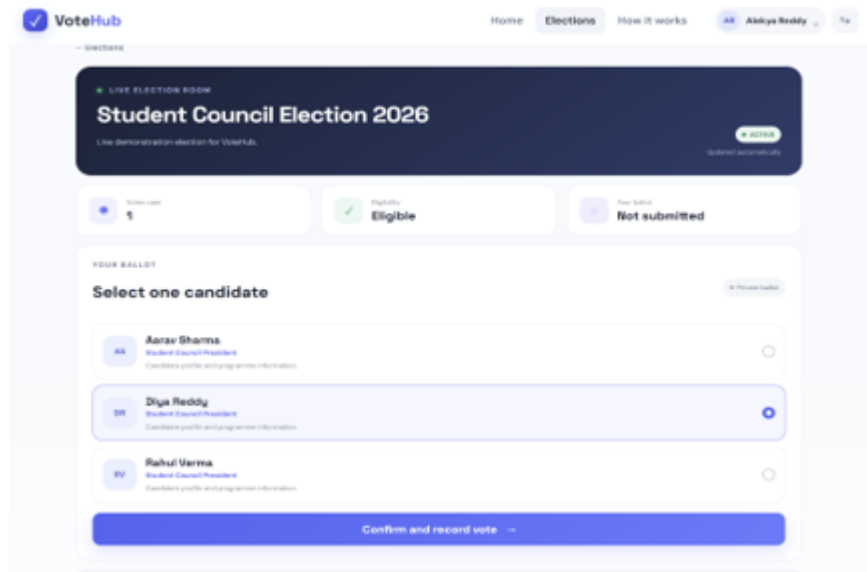
Voter dashboa

rd

Figure 14.1(e): Voter dashboard

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

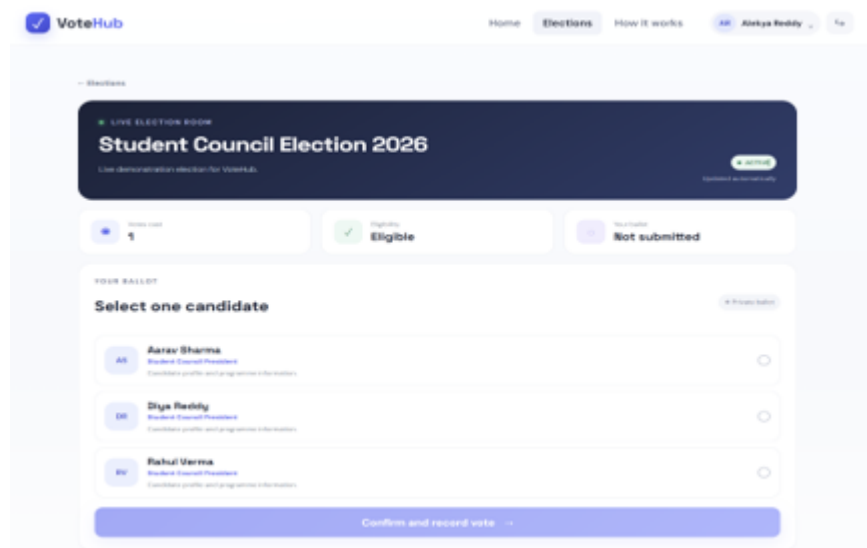
*Election list*



*Figure 14.1(f): Election list*

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

*Active election*



*Figure 14.1(g): Active election*

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

*Candidate selection*

The screenshot displays the 'VoteHub' interface for the 'Student Council Election 2026'. At the top, there's a navigation bar with 'Home', 'Elections', and 'How it works'. The 'Elections' tab is active. Below the navigation bar, the page title 'Student Council Election 2026' is shown, along with a 'LIVE ELECTION BOOM' banner and a 'Get started' button. The main content area is divided into three sections: 'Voting card' (showing '1'), 'Eligible' (with a green checkmark), and 'Not submitted' (with a red X). Below these, the 'YOUR BALLOT' section is titled 'Select one candidate'. It lists three candidates: 'Aarav Sharma' (Student Council President), 'Diya Reddy' (Student Council President), and 'Rahul Verma' (Student Council President). Each candidate has a radio button next to their name. At the bottom, there is a 'Confirm and record vote' button.

*Figure 14.1(h): Candidate selection*